

MICROSERVICES ARCHITECTURE

Implementation, API Testing & GitHub Submission Documentation

Spring Boot • REST APIs • Maven • Postman • Git • GitHub

Document Item	Details
Project	Microservice Architecture Project
Architecture	Microservices
Services	User Service, Product Service, Order Service
Testing Tool	Postman
Source Control	Git / GitHub
Repository	Microservice-architecture

1. Project Objective

The objective of this task is to implement and demonstrate a Spring Boot application using Microservices Architecture. The application is separated into independent services for users, products, and orders. Each service runs on a different port and exposes REST APIs.

The services were tested using Postman. The Order Service also demonstrates service-to-service communication by calling the User Service through RestTemplate.

2. Architecture Overview

The project contains three independent services: User Service on port 8081, Product Service on port 8082, and Order Service on port 8083.

The client/Postman sends requests to the appropriate service. The Order Service can call the User Service using `http://localhost:8081/users/{id}`.

3. Project Structure

```
microservices/  
  ■■■ user-service/  
  ■■■ product-service/  
  ■■■ order-service/  
  ■■■ monolith_vs_microservices_guide.docx
```

4. Technologies Used

Technology	Purpose
Java	Application programming language
Spring Boot	Building REST-based backend services
Maven	Dependency management and project build
REST API	Communication between client and services
RestTemplate	Order Service to User Service communication
Postman	API testing
Git / GitHub	Version control and remote repository
MySQL	Database technology where configured

5. User Service

User Service runs on port 8081.

Method	Endpoint	Purpose
GET	http://localhost:8081/users	Get users
GET	http://localhost:8081/users/{id}	Get a user by ID
POST	http://localhost:8081/users	Create a user
PUT	http://localhost:8081/users/{id}	Update a user
DELETE	http://localhost:8081/users/{id}	Delete a user

The User Service APIs were tested through Postman during implementation.

6. Product Service

Product Service runs on port 8082.

Method	Endpoint	Purpose
GET	http://localhost:8082/products	Get all products
GET	http://localhost:8082/products/{id}	Get a product by ID
POST	http://localhost:8082/products	Create a product
PUT	http://localhost:8082/products/{id}	Update a product
DELETE	http://localhost:8082/products/{id}	Delete a product

A successful Product Service GET test was performed for product ID 4. The response showed ID 4, product name “Gaming Laptop”, and price 65000.0.

7. Order Service

Order Service runs on port 8083.

Method	Endpoint	Purpose
GET	http://localhost:8083/orders	Check that Order Service is working
GET	http://localhost:8083/orders/user/{id}	Fetch user information through Order Service

7.1 OrderController Implementation

```
@RestController
public class OrderController {

    private final RestTemplate restTemplate = new RestTemplate();

    @GetMapping("/orders")
    public String getOrders() {
        return "Order Service is working";
    }

    @GetMapping("/orders/user/{id}")
    public ResponseEntity<String> getUserFromOrderService(@PathVariable Long id) {
        String url = "http://localhost:8081/users/" + id;
        ResponseEntity<String> response = restTemplate.getForEntity(url, String.class);
        return ResponseEntity.status(response.getStatusCode()).body(response.getBody());
    }
}
```

```
}
```

The current Order Service implementation does not require `Order.java` or `OrderRepository.java` because it is not storing orders in a database. Those classes would only be required if order persistence and CRUD operations were added.

8. API Testing Using Postman

8.1 Order Service Health Check

1. Open Postman. 2. Select GET. 3. Enter `http://localhost:8083/orders`. 4. Click Send. 5. Verify HTTP 200 OK and the response "Order Service is working".

8.2 Order Service → User Service Communication

1. Make sure User Service is running on port 8081. 2. Select GET. 3. Enter `http://localhost:8083/orders/user/3`. 4. Click Send. 5. Verify the user response is returned through the Order Service.

8.3 Product Service Test

Use GET `http://localhost:8082/products/4` and click Send. The recorded successful response was HTTP 200 OK with product ID 4, name "Gaming Laptop", and price 65000.0.

9. Troubleshooting

9.1 Port 8083 Already in Use

The terminal showed: "Web server failed to start. Port 8083 was already in use." This means another process was already listening on port 8083.

Resolution: stop the duplicate Order Service process, or configure the application to use another port. The intended configuration is:

```
server.port=8083
```

9.2 Git Modified and Untracked Files

Git can show modified and untracked files when changes have not been staged. The normal sequence is:

```
git add .
git status
git commit -m "Complete microservices project"
git push -u origin main
```

The project was successfully pushed to the GitHub main branch.

10. GitHub Repository

The completed source code was uploaded to the public GitHub repository:

```
https://github.com/shivasaikamaji/Microservice-architecture
```

The repository page shows order-service, product-service, user-service, and the project guide document.

11. Final Verification Checklist

- User Service configured on port 8081
- Product Service configured on port 8082
- Order Service configured on port 8083
- User Service APIs tested in Postman
- Product Service APIs tested in Postman
- GET /orders tested successfully
- GET /orders/user/3 tested
- Product GET /products/4 tested successfully
- Order Service → User Service communication implemented
- Git commit completed
- Git push completed
- GitHub repository verified

12. Conclusion

The project demonstrates a basic Microservices Architecture using Spring Boot. The application is divided into User, Product, and Order services, each running independently on its own port. REST APIs were tested using Postman, and the Order Service demonstrates communication with the User Service using RestTemplate.

The completed source code was committed and pushed to GitHub, making the project ready for repository-based review and submission.

13. Evidence Screenshots

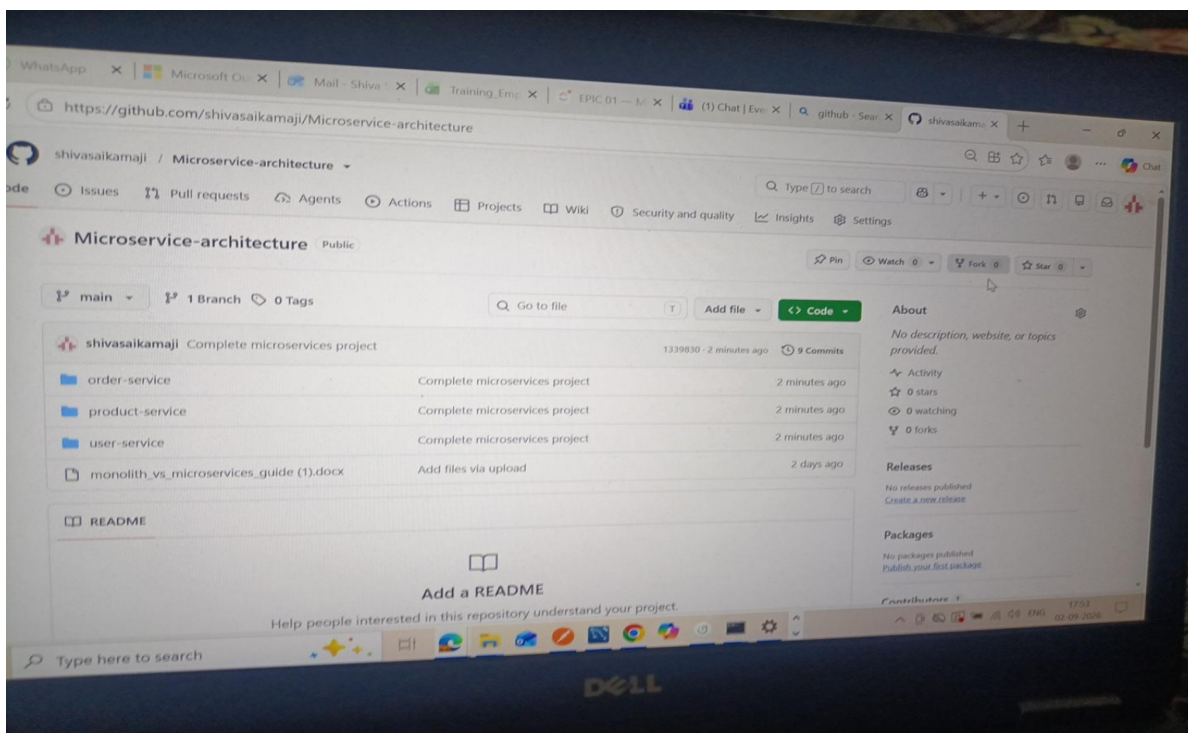


Figure 1. GitHub repository showing the three microservice folders and the guide document.

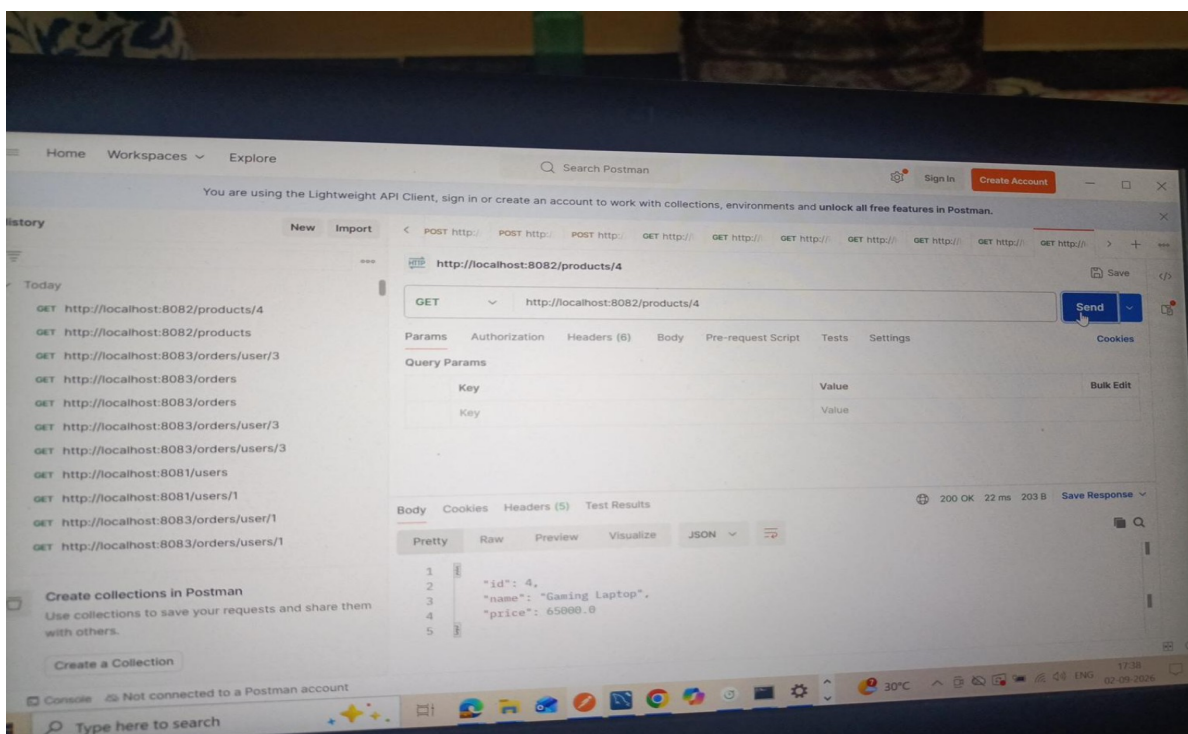


Figure 2. Postman Product Service GET request and successful JSON response.

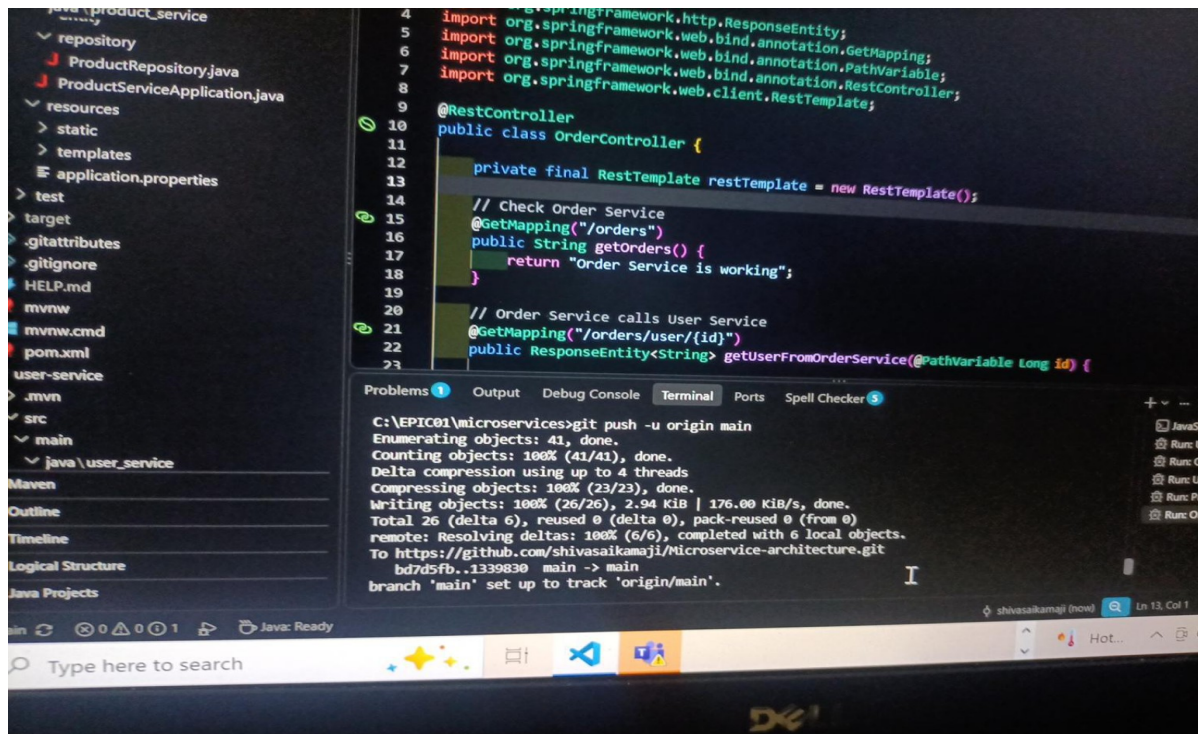


Figure 3. Git terminal output showing the successful push to the main branch.